

1 Interprocess communication

1.1 Producer-consumer buffer

Theorem 1.1. Suppose that the buffer starts empty, an insertion is permitted only when $n < N$, and a removal is permitted only when $n > 0$. If every counter update is protected by mutual exclusion, the number of elements remains between zero and the capacity N .

$$0 \leq n_{\text{buffer}} \leq N \quad (1.1)$$



Figure 1.1. Data flow through the shared buffer

Table 1.1. Buffer semaphore state

Semaphore	Initial value
mutex	1
empty	N
full	0

```

1 def can_consume(items):
2     return items > 0
  
```

Listing 1.1. Checking buffer availability

Input: A non-empty shared buffer

Output: The oldest element

```

1 wait(full)
2 wait(mutex)
3  $x \leftarrow$  remove element
4 signal(mutex)
5 signal(empty)
6 return  $x$ 
  
```

Algorithm 1.1. Removing an element from the buffer

The invariant in theorem 1.1, expressed by equation (1.1), prevents overflow and underflow. Figure 1.1 summarizes the flow, while table 1.1, listing 1.1, and algorithm 1.1 describe initialization and removal. Dijkstra's treatment of cooperating sequential processes includes the bounded-buffer construction used by this example (Dijkstra 1968).

Bibliography

Dijkstra, E. W. (1968). *Cooperating Sequential Processes (EWD 123)*. URL: <https://www.cs.utexas.edu/users/EWD/transcriptions/EWD01xx/EWD123.html> (visited on 08/04/2026).